NUCAMP · SOLO AI FOUNDER · WEEK 3-4

TaxNexus — CI/CD, Health, Backups & Restore

Tagged release pipeline · health endpoint · nightly MySQL backups · restore test

Project	**TaxNexus (soloai_starter)**
Repository	**darnell-ux/soloai_starter · main**
Release tag	**v1.0.0**
VPS	**Hostinger KVM2 · 2.25.203.69**
Domain	**taxnexusapp.com**
Verified	**2026-08-14 ·** PROD /health 200

§ Results at a glance

Obj	Deliverable	Status	Evidence
1	Tagged CI/CD release (`v*.*.*`)	BUILD GREEN DEPLOY: secrets	Build job + artifact succeeded; deploy job blocked on missing VPS_* secrets. Prod deployed manually.
2	`GET /health` → 200 JSON	LIVE 200	curl -I shows HTTP/2 200; `services.database: ok`
3	Nightly backups + retention	RUNNING	2 archives in /root/backups; cron @ 02:00; 7-day prune
4	Restore test → staging	PASSED	109/109 tables into taxnexus_staging; prod intact; /health 200

Action required — GitHub Actions secrets. `gh secret list` returns empty; the deploy job failed with `cd ""` (empty `VPS_APP_DIR`). Add these repo secrets (Settings → Secrets → Actions) so tagged deploys run automatically: `VPS_HOST=2.25.203.69`, `VPS_USER=root`, `VPS_APP_DIR=/root/soloai_starter`, and `VPS_SSH_KEY` (the deploy private key). Until then, production is deployed with `scripts/deploy.sh` over SSH (as done for this release).

1. Tagged CI/CD Release

New workflow triggering on annotated semantic-version tags. A secret-free **build** job compiles the app and uploads the artifact; a **deploy** job SSHes to the VPS and verifies `/health`.

.github/workflows/release.yml — on: push tags v*.*.*

```
name: release

# Tagged production release. Fires on annotated semantic-version tags (v*.*.*):
#   git tag -a v1.2.3 -m "..."   &&   git push origin v1.2.3
#
# Two jobs:
#   build  — compiles the SvelteKit app on the GitHub runner and uploads the
#            adapter-node output as a workflow artifact. Needs no secrets, so it
#            proves the release is buildable even before the VPS secrets exist.
#   deploy — SSHes into the Hostinger KVM2, resets to the tagged commit, rebuilds
#            the Docker image and recreates the app container, then verifies
#            /health returns 200. Requires the VPS_* repo secrets.
on:
  push:
    tags:
      - 'v*.*.*'

concurrency:
  group: release-production
  cancel-in-progress: false

permissions:
  contents: read

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout the tagged commit
        uses: actions/checkout@v4
```

```
      - name: Set up Node
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install dependencies (lockfile-exact)
        run: npm ci

      - name: Build the SvelteKit app
        # These ARGs/vars only satisfy the build-time env validator; no secrets are
        # baked. Real runtime values come from the VPS .env at container start.
        env:
          NODE_ENV: production
          PUBLIC_STRAPI_URL: https://cms.taxnexusapp.com
          STRAPI_API_URL: https://cms.taxnexusapp.com
          BETTER_AUTH_URL: https://taxnexusapp.com
          BETTER_AUTH_SECRET: build_time_placeholder_secret_min_32_characters
          DATABASE_URL: mysql://placeholder:placeholder@db:3306/soloai_db
        run: npm run build

      - name: Upload build artifact
        uses: actions/upload-artifact@v4
        with:
          name: taxnexus-build-${{ github.ref_name }}
          path: build/
          if-no-files-found: error
          retention-days: 14

  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment: production
    steps:
      - name: Deploy the tagged release over SSH
        uses: appleboy/ssh-action@v1.2.0
        with:
          host: ${{ secrets.VPS_HOST }}
          username: ${{ secrets.VPS_USER }}
          key: ${{ secrets.VPS_SSH_KEY }}
          script: |
            set -euo pipefail
            cd "${{ secrets.VPS_APP_DIR }}"
            echo "### Deploying tag ${{ github.ref_name }} ..."
            git fetch --all --tags --prune
            # Reset the working tree to the exact tagged commit.
            git checkout -f "${{ github.ref_name }}"
            git reset --hard "${{ github.ref_name }}"
            # Reuse the repo's idempotent deploy (build app+strapi, up -d, prune).
            DEPLOY_BRANCH="${{ github.ref_name }}" ./scripts/deploy.sh

      - name: Verify /health returns 200
        uses: appleboy/ssh-action@v1.2.0
        with:
          host: ${{ secrets.VPS_HOST }}
          username: ${{ secrets.VPS_USER }}
          key: ${{ secrets.VPS_SSH_KEY }}
          script: |
            set -euo pipefail
            echo "### Waiting for the app to become healthy ..."
            for i in $(seq 1 30); do
              code=$(curl -s -o /dev/null -w '%{http_code}' https://taxnexusapp.com/health || true)
              if [ "$code" = "200" ]; then
                echo "OK: /health returned 200 (attempt $i)"
                curl -s https://taxnexusapp.com/health
                exit 0
              fi
              echo "attempt $i: /health -> $code; retrying ..."
              sleep 4
            done
```

```
            echo "ERROR: /health never returned 200" >&2
            exit 1
```

Tag & push (Objective 1.4)

```
$ git tag -a v1.0.0 -m "TaxNexus v1.0.0 — initial production release"
$ git push origin v1.0.0
 * [new tag]         v1.0.0 -> v1.0.0
```

GitHub Actions run

run https://github.com/darnell-ux/soloai_starter/actions/runs/31836384634

```
build   -> success   (npm ci, npm run build, upload artifact: all green)
deploy  -> failure   step "Deploy the tagged release over SSH": cd "" (VPS_APP_DIR secret empty)
        (the deploy job is green once the VPS_* secrets are added)
```

> The **build job passing** proves the tagged commit is production-buildable on a clean runner. Production for v1.0.0 was rolled out manually via `scripts/deploy.sh` (git reset → rebuild → up -d) and verified below.

2. Health Check Endpoint

Contract: **always HTTP 200** — a degraded dependency shows in the body, never a 500. The DB probe runs SELECT 1 against the app's real datastore, the Better Auth SQLite DB (the SvelteKit app ships no MySQL client; MySQL belongs to Strapi/Mautic).

src/routes/health/+server.ts — SvelteKit +server

```
import { json } from '@sveltejs/kit';
import type { RequestHandler } from './$types';
import { openAuthDatabase } from '$lib/server/auth-options';

/**
 * Liveness/readiness probe for uptime monitors, the release workflow, and load
 * balancers. Contract: **always HTTP 200** with a JSON body — never 500 — so a
 * degraded dependency surfaces in the payload (`services.*` + top-level `status`)
 * instead of tripping a naive `-I` check into a hard failure.
 *
 * Version is pinned to the release tag; bump it alongside package.json on release.
 *
 * Database check: a trivial `SELECT 1` against the app's real datastore — the
 * Better Auth SQLite database (better-sqlite3). The SvelteKit app has no MySQL
 * client; MySQL belongs to Strapi/Mautic. Checking SQLite verifies the store the
 * app actually reads/writes on every authenticated request.
 */
const APP_VERSION = '1.0.0';

function checkDatabase(): 'ok' | 'degraded' {
        let db: ReturnType<typeof openAuthDatabase> | undefined;
        try {
                db = openAuthDatabase();
                const row = db.prepare('SELECT 1 AS ok').get() as { ok?: number } | undefined;
                return row?.ok === 1 ? 'ok' : 'degraded';
        } catch {
                // Never propagate — a dead DB must not turn the probe into a 500.
                return 'degraded';
        } finally {
                try {
                        db?.close();
                } catch {
                        /* ignore close errors */
                }
```

```
        }
}

export const GET: RequestHandler = () => {
        const database = checkDatabase();
        const status = database === 'ok' ? 'ok' : 'degraded';

        return json(
                {
                        status,
                        version: APP_VERSION,
                        timestamp: new Date().toISOString(),
                        services: {
                                app: 'ok',
                                database
                        }
                },
                {
                        status: 200,
                        headers: {
                                // Probes must never be served from a cache.
                                'cache-control': 'no-store'
                        }
                }
        );
};

// Health must be evaluated per-request, never prerendered to a static asset.
export const prerender = false;
```

Production verification (Objective 2.6)

curl against https://taxnexusapp.com/health — live

```
TaxNexus — Objective 2 Health Check — production verification
Captured: 2026-08-14 13:11:12 PDT
Host: taxnexusapp.com (VPS 2.25.203.69)
============================================================

$ curl -I https://taxnexusapp.com/health
HTTP/2 200
server: nginx/1.31.1
date: Fri, 14 Aug 2026 20:11:13 GMT
content-type: application/json
content-length: 112
cache-control: no-store
strict-transport-security: max-age=31536000; includeSubDomains


$ curl https://taxnexusapp.com/health
{"status":"ok","version":"1.0.0","timestamp":"2026-08-14T20:11:13.521Z","services":{"app":"ok","database":"ok"}}
```

HTTP/2 200, `cache-control: no-store`, `{"status":"ok", …, "services":{"app":"ok","database":"ok"}}`.

3. Nightly Backups

Dumps all MySQL databases (mautic + strapi_db + grants) via `docker exec`, timestamped gzip archive to `/root/backups`, 7-day retention. **No secrets in the script** — the root password is read inside the container from its own `MYSQL_ROOT_PASSWORD`.

scripts/backup.sh — mysqldump · docker exec

```
#!/usr/bin/env bash
#
# Nightly MySQL backup for TaxNexus (Hostinger KVM2).
#
```

```
# Dumps the Dockerized MySQL server (all databases: mautic + strapi_db + grants)
# to a timestamped, gzip-compressed archive, then prunes archives older than the
# retention window.
#
# NO SECRETS IN THIS SCRIPT. The MySQL root password is read *inside* the db
# container from its own MYSQL_ROOT_PASSWORD environment variable (exported to
# MYSQL_PWD for the client), so it never appears in this file, in the host
# process list, or in shell history.
#
# Cron (installed on the VPS):
#   0 2 * * * /root/soloai_starter/scripts/backup.sh >> /var/log/taxnexus-backup.log 2>&1
#
# Manual run:
#   bash scripts/backup.sh
#
# Overridable via environment:
#   BACKUP_DIR      (default /root/backups)
#   DB_CONTAINER    (default taxnexus-db-1)
#   RETENTION_DAYS  (default 7)
set -euo pipefail

BACKUP_DIR="${BACKUP_DIR:-/root/backups}"
DB_CONTAINER="${DB_CONTAINER:-taxnexus-db-1}"
RETENTION_DAYS="${RETENTION_DAYS:-7}"

timestamp="$(date +%Y%m%d-%H%M)"
archive="${BACKUP_DIR}/taxnexus-${timestamp}.sql.gz"
tmp="${archive}.partial"

log() { echo "[$(date '+%Y-%m-%d %H:%M:%S')] $*"; }

# Preflight: the db container must be running.
if ! docker ps --format '{{.Names}}' | grep -qx "${DB_CONTAINER}"; then
  echo "ERROR: db container '${DB_CONTAINER}' is not running." >&2
  exit 1
fi

mkdir -p "${BACKUP_DIR}"

log "Starting backup -> ${archive}"

# Consistent (InnoDB) snapshot of everything. The password is expanded inside the
# container via MYSQL_PWD; nothing sensitive crosses the host boundary.
set -o pipefail
docker exec "${DB_CONTAINER}" sh -c '
  MYSQL_PWD="$MYSQL_ROOT_PASSWORD" exec mysqldump \
    -uroot \
    --all-databases \
    --single-transaction \
    --quick \
    --routines \
    --triggers \
    --events \
    --default-character-set=utf8mb4
' | gzip -c > "${tmp}"

# Guard against a truncated/empty dump before we publish the final name.
if [ ! -s "${tmp}" ]; then
  rm -f "${tmp}"
  echo "ERROR: dump produced an empty archive; aborting." >&2
  exit 1
fi

mv "${tmp}" "${archive}"
size="$(du -h "${archive}" | cut -f1)"
log "Backup complete: ${archive} (${size})"

# Retention: delete archives older than RETENTION_DAYS days.
log "Pruning backups older than ${RETENTION_DAYS} day(s) in ${BACKUP_DIR}"
deleted="$(find "${BACKUP_DIR}" -maxdepth 1 -name 'taxnexus-*.sql.gz' -type f -mtime "+${RETENTION_DAYS}" -
print -delete | wc -l | tr -d ' ')"
```

```
log "Pruned ${deleted} old archive(s)"

log "Current backups:"
ls -la "${BACKUP_DIR}"
```

Cron install (Objective 3.3)

```
$ crontab -l
0 2 * * * /root/soloai_starter/scripts/backup.sh >> /var/log/taxnexus-backup.log 2>&1
```

Manual backup proof (Objective 3.5)

on the VPS: bash scripts/backup.sh && ls -la /root/backups/ — live

```
TaxNexus — Objective 3 Nightly Backup — VPS proof
Captured: 2026-08-14 20:11:21 UTC  Host: 2.25.203.69 (/root/soloai_starter)
============================================================

$ bash scripts/backup.sh
[2026-08-14 20:11:21] Starting backup -> /root/backups/taxnexus-20260814-2011.sql.gz
[2026-08-14 20:11:22] Backup complete: /root/backups/taxnexus-20260814-2011.sql.gz (980K)
[2026-08-14 20:11:22] Pruning backups older than 7 day(s) in /root/backups
[2026-08-14 20:11:22] Pruned 0 old archive(s)
[2026-08-14 20:11:22] Current backups:
total 1916
drwxr-xr-x  2 root root    4096 Aug 14 20:11 .
drwx------ 12 root root    4096 Aug 14 20:07 ..
-rw-r--r--  1 root root  949113 Aug 14 20:07 taxnexus-20260814-2007.sql.gz
-rw-r--r--  1 root root 1003483 Aug 14 20:11 taxnexus-20260814-2011.sql.gz

$ ls -la /root/backups/
total 1916
drwxr-xr-x  2 root root    4096 Aug 14 20:11 .
drwx------ 12 root root    4096 Aug 14 20:07 ..
-rw-r--r--  1 root root  949113 Aug 14 20:07 taxnexus-20260814-2007.sql.gz
-rw-r--r--  1 root root 1003483 Aug 14 20:11 taxnexus-20260814-2011.sql.gz

$ crontab -l   # nightly 02:00 job
0 2 * * * /root/soloai_starter/scripts/backup.sh >> /var/log/taxnexus-backup.log 2>&1
```

3b. Restore Script

Two modes: production-safe staging (single-db) load used by the restore test, and a `CONFIRM=RESTORE`-guarded full disaster-recovery restore. Verifies with a `SELECT COUNT`.

scripts/restore.sh — staging + full DR

```
#!/usr/bin/env bash
#
# Restore a TaxNexus MySQL backup produced by scripts/backup.sh.
#
# Two modes:
#
#   1) STAGING / single-database (SAFE, default when a target db is given):
#        bash scripts/restore.sh <archive.sql.gz> <target_db>
#      Extracts one source database from the all-databases dump and loads it into
#      <target_db>, creating that schema if needed. Production databases are left
#      untouched. This is the mode used by the restore *test* (target
#      taxnexus_staging). Choose the source with SOURCE_DB (default: mautic).
#
#   2) FULL disaster recovery (DESTRUCTIVE — overwrites production):
#        CONFIRM=RESTORE bash scripts/restore.sh <archive.sql.gz>
#      Streams the entire dump back into the live server, recreating every
#      database exactly as captured. Requires CONFIRM=RESTORE to run.
```

```
#
# NO SECRETS IN THIS SCRIPT — same MYSQL_PWD-inside-the-container pattern as
# backup.sh.
#
# Overridable via environment:
#   DB_CONTAINER  (default taxnexus-db-1)
#   SOURCE_DB     (default mautic)  — which db to lift out of the dump in mode 1
set -euo pipefail

DB_CONTAINER="${DB_CONTAINER:-taxnexus-db-1}"
SOURCE_DB="${SOURCE_DB:-mautic}"

archive="${1:-}"
target_db="${2:-}"

log() { echo "[$(date '+%Y-%m-%d %H:%M:%S')] $*"; }

usage() {
  echo "Usage:" >&2
  echo "  bash scripts/restore.sh <archive.sql.gz> <target_db>          # staging/single-db (safe)" >&2
  echo "  CONFIRM=RESTORE bash scripts/restore.sh <archive.sql.gz>      # full DR (overwrites prod)" >&2
  exit 2
}

[ -n "${archive}" ] || usage
if [ ! -f "${archive}" ]; then
  echo "ERROR: archive not found: ${archive}" >&2
  exit 1
fi
if ! docker ps --format '{{.Names}}' | grep -qx "${DB_CONTAINER}"; then
  echo "ERROR: db container '${DB_CONTAINER}' is not running." >&2
  exit 1
fi

# Run a mysql client statement inside the container (password via MYSQL_PWD).
mysql_exec() {
  docker exec -i "${DB_CONTAINER}" sh -c 'MYSQL_PWD="$MYSQL_ROOT_PASSWORD" exec mysql -uroot "$@"' _ "$@"
}

if [ -n "${target_db}" ]; then
  # ---- Mode 1: staging / single-database restore (production-safe) ----------
  log "STAGING restore: '${SOURCE_DB}' (from ${archive}) -> schema '${target_db}'"

  log "Creating target schema '${target_db}' if absent ..."
  mysql_exec -e "CREATE DATABASE IF NOT EXISTS \`${target_db}\` CHARACTER SET utf8mb4;"

  # Slice the single source database out of the --all-databases dump and strip its
  # CREATE DATABASE / USE lines so the tables land in <target_db> instead of prod.
  # We prepend FOREIGN_KEY_CHECKS=0: mysqldump's FK-disable pragma lives in the
  # file header (before the first database), which the slice deliberately skips —
  # without it, tables with cross-references (e.g. mautic's `leads`) fail to load
  # in dump order. Re-enabled at the end so the restored schema stays consistent.
  log "Loading '${SOURCE_DB}' tables into '${target_db}' ..."
  {
    echo 'SET FOREIGN_KEY_CHECKS=0;'
    echo 'SET UNIQUE_CHECKS=0;'
    gunzip -c "${archive}" \
      | sed -n "/^-- Current Database: \`${SOURCE_DB}\`/,/^-- Current Database: \`/p" \
      | grep -vE '^(CREATE DATABASE|USE )'
    echo 'SET FOREIGN_KEY_CHECKS=1;'
    echo 'SET UNIQUE_CHECKS=1;'
  } | docker exec -i "${DB_CONTAINER}" sh -c 'MYSQL_PWD="$MYSQL_ROOT_PASSWORD" exec mysql -uroot "$1"' _
"${target_db}"

  # Verify with a SELECT COUNT.
  table_count="$(mysql_exec -N -B -e \
    "SELECT COUNT(*) FROM information_schema.tables WHERE table_schema='${target_db}';")"
  log "Verification: schema '${target_db}' now has ${table_count} table(s)."
  if [ "${table_count}" -gt 0 ]; then
    log "RESTORE TEST PASSED — staging schema populated, production untouched."
  else
```

```
    echo "ERROR: staging schema '${target_db}' has no tables after restore." >&2
    exit 1
  fi
else
  # ---- Mode 2: full disaster-recovery restore (DESTRUCTIVE) -----------------
  if [ "${CONFIRM:-}" != "RESTORE" ]; then
    echo "REFUSING full restore without CONFIRM=RESTORE (this overwrites production)." >&2
    usage
  fi
  log "FULL restore from ${archive} into live server (all databases) ..."
  gunzip -c "${archive}" \
    | docker exec -i "${DB_CONTAINER}" sh -c 'MYSQL_PWD="$MYSQL_ROOT_PASSWORD" exec mysql -uroot'
  count="$(mysql_exec -N -B -e 'SELECT COUNT(*) FROM information_schema.schemata;')"
  log "Verification: server now has ${count} schema(s). FULL restore complete."
fi
```

4. Restore Test → Staging

Restores the nightly archive into an isolated `taxnexus_staging` schema — never production — verifies table/row counts, and confirms production `/health` is unaffected before and after. Full sequence with timestamps:

reference-launch/week3-restore-proof.txt — live · timestamped

```
TaxNexus — Objective 4 Restore Test (to staging, prod-safe)
Captured: 2026-08-14 20:11:40 UTC  Host: 2.25.203.69
Archive under test: /root/backups/taxnexus-20260814-2011.sql.gz
============================================================

# Reset any prior test schema so this run is clean
[2026-08-14 20:11:40 UTC] $ mysql -e "DROP DATABASE IF EXISTS taxnexus_staging"

[2026-08-14 20:11:41 UTC] PROD databases + /health BEFORE restore:
information_schema mautic mysql performance_schema soloai_db strapi_db sys
  prod /health -> HTTP 200

[2026-08-14 20:11:41 UTC] $ SOURCE_DB=mautic bash scripts/restore.sh "/root/backups/taxnexus-20260814-
2011.sql.gz" taxnexus_staging
[2026-08-14 20:11:41] STAGING restore: 'mautic' (from /root/backups/taxnexus-20260814-2011.sql.gz) -> schema
'taxnexus_staging'
[2026-08-14 20:11:41] Creating target schema 'taxnexus_staging' if absent ...
[2026-08-14 20:11:41] Loading 'mautic' tables into 'taxnexus_staging' ...
[2026-08-14 20:11:44] Verification: schema 'taxnexus_staging' now has 109 table(s).
[2026-08-14 20:11:44] RESTORE TEST PASSED — staging schema populated, production untouched.

[2026-08-14 20:11:44 UTC] SELECT COUNT verification (tables per schema):
mautic  109
strapi_db       52
taxnexus_staging        109

[2026-08-14 20:11:44 UTC] Row-count spot check: SELECT COUNT(*) FROM taxnexus_staging.leads;
50

[2026-08-14 20:11:44 UTC] PROD databases + /health AFTER restore (must be unaffected):
information_schema mautic mysql performance_schema soloai_db strapi_db sys taxnexus_staging
{"status":"ok","version":"1.0.0","timestamp":"2026-08-14T20:11:44.572Z","services":{"app":"ok","database":"ok"}}
prod /health body: 200
```

> **RESTORE TEST PASSED** — `taxnexus_staging` restored **109/109** tables (matching prod `mautic`), `leads` = 50 rows; prod `mautic`/`strapi_db` unchanged; prod `/health` = 200 before and after.

TaxNexus — Week 3-4 Nucamp Submission · Generated 2026-08-14 · Artifacts: week3-health-check.txt · week3-backup-proof.txt · week3-restore-proof.txt · Release v1.0.0 · Verified live against production (2.25.203.69 / taxnexusapp.com).